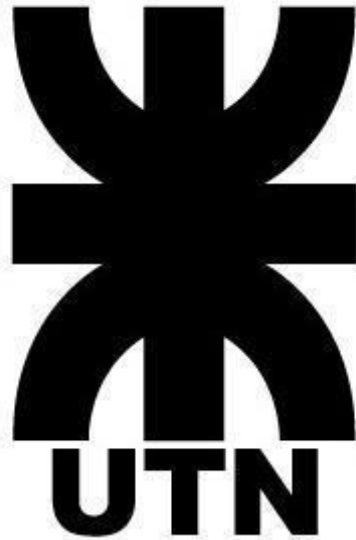


# Universidad Tecnológica Nacional



**Cátedra: Algoritmos Genéticos**

**3º Año Ingeniería en Sistemas de Información**

**Trabajo Práctico Integrador**

**Optimización del despliegue de Reactores Modulares Pequeños**

**Ciclo lectivo: 2026**

**Comisión: 303**

**Grupo : 14**

**Integrantes:**

|                       |       |                          |
|-----------------------|-------|--------------------------|
| - RIBOTTA, Tomás      | 52309 | tomasribotta20@gmail.com |
| - MAZALAN, Ariel      | 52867 | arielmazalan11@gmail.com |
| - GIACONE, Alessandro | 52664 | alef.giacone@gmail.com   |
| - BUSNADIEGO, Martin  | 53020 | martinbusnaisi@gmail.com |

|                                                         |    |
|---------------------------------------------------------|----|
| 1. Introducción y alcance del prototipo.....            | 3  |
| 2. Diseño del modelo.....                               | 3  |
| 2.1 Representación de los individuos.....               | 3  |
| 2.2 Función de fitness.....                             | 3  |
| 2.3 Operadores y parámetros del algoritmo genético..... | 4  |
| 3. Datos utilizados.....                                | 4  |
| 4. Estructura del código.....                           | 5  |
| 4.1 Funciones geográficas.....                          | 5  |
| 4.2 Densidad poblacional (GHS-POP).....                 | 6  |
| 4.3 Costo de construcción.....                          | 6  |
| 4.4 Algoritmo genético.....                             | 7  |
| 4.5 Reporte de resultados.....                          | 8  |
| 5. Especificación técnica e infraestructura.....        | 8  |
| 5.1 Software.....                                       | 8  |
| 5.2 Hardware e infraestructura.....                     | 8  |
| 5.3 Ejecución.....                                      | 9  |
| 6. Salidas del prototipo.....                           | 9  |
| 7. Resultados y verificación.....                       | 9  |
| Relación con los objetivos del proyecto.....            | 10 |
| 8. Limitaciones y trabajo futuro.....                   | 11 |
| Referencias.....                                        | 12 |
| Anexo: Código del Trabajo Práctico.....                 | 13 |

## 1. Introducción y alcance del prototipo

Esta segunda parte describe el prototipo de software que traslada el marco teórico de la primera parte a un modelo ejecutable. El problema es elegir, dentro de un corredor ribereño que va desde el Río Paraná hasta la costa bonaerense, la ubicación de una central nuclear modular pequeña (SMR) que combine bajo costo de construcción con baja población expuesta.

El prototipo implementa las variables desarrolladas en el marco teórico: costo del tendido eléctrico (sección 4.4), costo de envío de materiales (4.5) y densidad poblacional por anillos de planificación de emergencia (4.3). El riesgo sísmico (4.2) no forma parte del fitness: todo el corredor está en la Zona 0 del reglamento INPRES-CIRSOC 103, por lo que no permite diferenciar un punto de otro.

El código está escrito en Python, en un único archivo (ga\_central\_nuclear.py), y puede reutilizarse como base para continuar la investigación con otros criterios, otros corredores o distintos valores de los parámetros.

## 2. Diseño del modelo

### 2.1 Representación de los individuos

Cada individuo se representa con un único número real  $t$  en  $[0, 1]$ . La función `interpolate_point` traduce  $t$  a un punto (lat, lon) sobre una polilínea de 40 vértices que recorre la margen del Paraná, el Río de la Plata y la costa hacia el sudeste:  $t = 0$  es el vértice norte y  $t = 1$  el vértice sur, y los valores intermedios se ubican proporcionalmente sobre el tramo correspondiente.

Esta codificación tiene dos ventajas. Reduce el espacio de búsqueda de dos dimensiones (latitud y longitud) a una, y garantiza que todo individuo sea un sitio válido por construcción, ya que siempre tiene acceso al agua de refrigeración que necesita la central, sin necesidad de agregar una restricción que descarte puntos tierra adentro.

### 2.2 Función de fitness

El fitness combina dos objetivos mediante una suma ponderada, de modo de poder usar selección, cruce y mutación estándar sobre un único valor escalar.

**Costo de construcción.** Para un punto  $p$ , el costo bruto es la suma del tendido eléctrico hasta la estación transformadora más cercana y del envío de materiales desde el puerto más cercano. Las distancias se calculan con la fórmula de Haversine ( $R = 6371$  km):

- $\text{costo\_tendido}(p) = d_{\text{red}}(p) \cdot 300.000 \text{ USD/km}$
- $\text{costo\_envio}(p) = d_{\text{puerto}}(p) \cdot 5.000 \text{ USD/km}$
- $\text{costo\_bruto}(p) = \text{costo\_tendido}(p) + \text{costo\_envio}(p)$

Los costos brutos de la población se normalizan con min-max y el fitness de costo es  $f_{\text{costo}} = 1 - \text{costo normalizado}$ , de manera que 1 corresponde al sitio más barato de la generación y 0 al más caro.

**Densidad poblacional.** Para cada punto se estima la densidad máxima (peor caso) de dos anillos concéntricos, muestreando radios y ángulos alrededor del sitio sobre el raster GHS-POP: el anillo de 0 a 2 millas (restricción dura, 6 radios  $\times$  20 ángulos) y el de 2 a 10

millas (restricción blanda, 5 radios × 16 ángulos). Se usa el máximo y no el promedio porque lo relevante es que exista algún punto de densidad peligrosa dentro del anillo, algo que un promedio podría ocultar.

Cada anillo aporta un puntaje entre 0 y 1: vale 1 si la densidad máxima no supera el umbral y decae exponencialmente a medida que el exceso crece, para que el algoritmo tenga un gradiente que seguir en lugar de un salto a cero.

- $f_1 = 1$  si  $d_1 \leq 25$ ;  $f_1 = \exp(-8 \cdot (d_1 - 25) / 25)$  si  $d_1 > 25$
- $f_2 = 1$  si  $d_2 \leq 400$ ;  $f_2 = \exp(-(d_2 - 400) / 400)$  si  $d_2 > 400$
- $f_{\text{poblacional}} = 0,65 \cdot f_1 + 0,35 \cdot f_2$

**Fitness total.**  $f_{\text{total}} = 0,5 \cdot f_{\text{costo}} + 0,5 \cdot f_{\text{poblacional}}$ . Si el sitio viola la restricción dura ( $d_1 > 25$  hab/km<sup>2</sup>), el fitness total se multiplica por 0,02. La penalización es multiplicativa y no una eliminación (fitness = 0) para que el algoritmo conserve información sobre qué dirección seguir para salir de la zona prohibida, y es lo bastante fuerte como para que ningún ahorro de costo compense el incumplimiento.

### 2.3 Operadores y parámetros del algoritmo genético

| Componente           | Valor                        | Detalle                                                                                 |
|----------------------|------------------------------|-----------------------------------------------------------------------------------------|
| Tamaño de población  | 60                           | Inicialización uniforme de t en [0, 1]                                                  |
| Generaciones         | 100                          | Criterio de corte fijo                                                                  |
| Selección            | Torneo, k = 2                | Se elige el mejor de dos individuos al azar                                             |
| Cruza                | Aritmética, p = 0,75         | hijo1 = $\alpha \cdot t_1 + (1-\alpha) \cdot t_2$ , hijo2 simétrico, $\alpha$ aleatorio |
| Mutación             | Gaussiana, p = 0,25          | $\sigma = 0,25$ ; t se acota a [0, 1]                                                   |
| Reemplazo            | Generacional                 | Sin elitismo; se guarda el mejor individuo global encontrado                            |
| Pesos del fitness    | costo 0,5 / población 0,5    | Dentro de población: anillo duro 0,65, anillo blando 0,35                               |
| Umbrales de densidad | 25 / 400 hab/km <sup>2</sup> | Anillo 0-2 millas / anillo 2-10 millas                                                  |
| Penalización         | 0,02                         | Factor si se viola la restricción dura                                                  |

Tabla 1. Parámetros del algoritmo genético

### 3. Datos utilizados

| Dato                 | Fuente / valor                                                                                               | Uso                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Densidad poblacional | GHS-POP R2023A, escenario E2030, Comisión Europea / JRC; celdas de 1 km en proyección Mollweide (ESRI:54009) | Fitness poblacional y capa del mapa |

| Dato                       | Fuente / valor                                                                                       | Uso                                   |
|----------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------|
| Corredor                   | 40 vértices (lat, lon) trazados sobre la ribera del Paraná, el Río de la Plata y la costa bonaerense | Espacio de búsqueda (t → punto)       |
| Estaciones transformadoras | 5 estaciones de la red de alta tensión (Resistencia, Santo Tomé, Rosario Oeste, Ezeiza, Necochea)    | Costo del tendido                     |
| Puertos                    | 7 puertos (Buenos Aires, Rosario, La Plata, Zárate, Santa Fe, Quequén/Necochea, San Nicolás)         | Costo de envío                        |
| Ciudades y aeropuertos     | 8 ciudades con población aproximada y 5 aeropuertos                                                  | Referencia visual en el mapa          |
| Costos unitarios           | 300.000 USD/km de línea; 5.000 USD/km de transporte                                                  | Valores de referencia (ver sección 8) |

*Tabla 2. Datos de entrada del modelo*

Los tiles de GHS-POP se leen desde la carpeta ghs\_pop\_tiles/ ubicada junto al script y se combinan en un mosaico en memoria. Para este corredor se usan los tiles R13\_C12, R13\_C13, R14\_C12 y R14\_C13. La proyección Mollweide se define explícitamente en el código en lugar de usar la embebida en los archivos, porque con algunas versiones de PROJ/GDAL el código ESRI:54009 no se resuelve bien y el raster aparece desplazado.

#### 4. Estructura del código

Esta sección describe cada función del script en el orden en que aparecen, indicando su firma, qué recibe, qué hace internamente y qué devuelve.

##### 4.1 Funciones geográficas

**haversine(p1, p2)** recibe dos puntos  $p1 = (lat1, lon1)$  y  $p2 = (lat2, lon2)$  en grados y devuelve la distancia entre ambos en kilómetros sobre la superficie terrestre. Convierte las cuatro coordenadas a radianes, calcula las diferencias  $\Delta\phi$  y  $\Delta\lambda$ , aplica la fórmula de Haversine ( $a = \sin^2(\Delta\phi/2) + \cos(\phi1) \cdot \cos(\phi2) \cdot \sin^2(\Delta\lambda/2)$ ;  $distancia = 2 \cdot R \cdot \text{asin}(\sqrt{a})$ ) con  $R = 6371$  km, y devuelve ese valor. Se usa en todo el script cada vez que hay que medir una distancia: al punto del corredor, a una estación transformadora o a un puerto.

**destino(point, bearing\_deg, distancia\_km)** recibe un punto de partida, un rumbo en grados ( $0^\circ$  = norte,  $90^\circ$  = este) y una distancia en kilómetros, y devuelve el punto (lat, lon) al que se llega. Usa las fórmulas directas de navegación esférica sobre una Tierra de radio  $R = 6371$  km. Esta función es la que permite "salir" del punto candidato en distintas direcciones para muestrear los anillos de densidad poblacional: `densidad_maxima_en_anillo` la llama una vez por cada combinación de radio y ángulo.

**interpolate\_point(t, boundary\_points)** es la función que traduce el cromosoma del algoritmo genético (un único número  $t$ ) en una coordenada real. Primero acota  $t$  a  $[0, 1]$ . Con  $n =$  la cantidad de vértices de la polilínea (40 en este proyecto), calcula  $idx\_float = t \cdot (n - 1)$ ; la parte entera  $i$  indica entre qué dos vértices cae  $t$ , y la parte fraccionaria  $frac$  indica en qué proporción del tramo. Si  $i$  llega al último índice devuelve directamente el último vértice; si no, interpola linealmente latitud y longitud por separado entre `boundary_points[i]` y

boundary\_points[i+1]. El resultado es que  $t$  recorre continuamente todo el corredor, del extremo norte ( $t = 0$ ) al extremo sur ( $t = 1$ ), sin poder salirse nunca de él.

#### 4.2 Densidad poblacional (GHS-POP)

**\_obtener\_raster\_ghs\_pop()** es la función que carga el raster de población y lo deja listo para consultarlo por coordenadas. La primera vez que se llama, busca todos los archivos .tif de la carpeta ghs\_pop\_tiles/, los abre con rasterio y los combina en un único mosaico en memoria con rasterio.merge.merge. En vez de confiar en el sistema de coordenadas que traen los .tif (que en algunas combinaciones se interpreta mal), fuerza explícitamente la proyección Mollweide estándar (GHS\_POP\_PROJ4) al escribir el mosaico en un rasterio.io.MemoryFile. También arma un pyproj.Transformer de WGS84 (lat/lon) a esa proyección, que es el que usa el resto del código para pasar de coordenadas geográficas a coordenadas del raster. Como abrir y mezclar los tiles es costoso, el resultado se guarda en variables globales (\_ghs\_pop\_dataset, \_transformer\_wgs84\_a\_mollweide) y las llamadas siguientes lo devuelven directamente sin volver a leer los archivos.

**densidad\_en\_puntos(points)** recibe una lista de puntos (lat, lon) y devuelve la densidad poblacional (hab/km<sup>2</sup>) de cada uno. Transforma todas las coordenadas de una sola vez a la proyección del raster y usa dataset.sample(...) de rasterio para leer el valor de la celda que le corresponde a cada punto. Como cada celda de GHS-POP mide 1 km × 1 km, el valor de la celda (población estimada de esa celda) equivale directamente a la densidad en hab/km<sup>2</sup>. Los valores nulos o por debajo de GHS\_POP\_NODATA (agua, zonas sin dato) se devuelven como 0.

**densidad\_maxima\_en\_anillo(point, r\_min\_km, r\_max\_km, n\_radios, n\_angulos)** estima la densidad más alta que puede encontrarse en un anillo alrededor de point, entre r\_min\_km y r\_max\_km de distancia. Genera una grilla de radios (con np.linspace) y de ángulos (0° a 360°, sin repetir el último) y, para cada combinación, calcula el punto correspondiente con destino(); a esa lista le agrega el propio centro del sitio. Con n\_radios × n\_angulos por defecto (más el centro) arma un conjunto de puntos de muestra, consulta la densidad de todos ellos con una sola llamada a densidad\_en\_puntos y devuelve el máximo. Se usa el máximo, y no el promedio, a propósito: interesa detectar si existe algún punto puntualmente denso dentro del anillo, aunque el resto esté vacío.

**evaluar\_restriccion\_poblacional(point)** arma el criterio poblacional completo del fitness. Llama dos veces a densidad\_maxima\_en\_anillo: una para el anillo 0-2 millas (con más resolución, 6 radios × 20 ángulos, porque decide si el sitio es viable) y otra para el anillo 2-10 millas (5 × 16). Compara la densidad de la zona dura contra UMBRAL\_DENSIDAD\_BAJA (25 hab/km<sup>2</sup>): si no la supera, cumple\_restriccion\_dura es True y f\_zona\_baja = 1; si la supera, f\_zona\_baja decae exponencialmente con  $\exp(-8 \cdot \text{exceso\_relativo})$ , donde exceso\_relativo es cuánto se pasó del umbral en proporción a él mismo. Hace lo mismo para la zona media contra UMBRAL\_DENSIDAD\_MEDIA (400 hab/km<sup>2</sup>), con un decaimiento más suave (exponente 1 en vez de 8). Combina ambos puntajes con los pesos 0,65 y 0,35 y devuelve una tupla (fitness, cumple\_restriccion\_dura, detalle), donde detalle es un diccionario con las densidades y puntajes de cada zona, usado después para los reportes y el popup del mapa.

#### 4.3 Costo de construcción

**costo\_tendido\_electrico(point)** calcula, con haversine, la distancia de point a cada una de las 5 estaciones transformadoras de ESTACIONES\_TRANSFORMADORAS, se queda con

la mínima ( $\min(\dots)$ ) y la multiplica por `COSTO_TENDIDO_USD_KM` (300.000 USD/km). Devuelve el costo en dólares de tender la línea de alta tensión hasta el nodo de la red más cercano.

**costo\_envio\_materiales(point)** hace exactamente el mismo cálculo pero contra la lista `PUERTOS`, usando `COSTO_TRANSPORTE_MARITIMO_USD_KM` (5.000 USD/km), para estimar el costo de transporte marítimo de los módulos desde el puerto más cercano.

**costo\_construccion\_bruto(point)** simplemente suma los dos resultados anteriores. Es la función que usa `evaluar_poblacion` para obtener el costo total en USD de cada individuo antes de normalizarlo.

**normalizar(valores)** aplica la normalización min-max estándar: para una lista de valores, resta el mínimo y divide por el rango (máximo - mínimo), protegiendo la división contra rangos casi nulos con un epsilon ( $1e-9$ ). El resultado es una lista con el mismo orden, donde 0 corresponde al valor más chico del lote y 1 al más grande. Se aplica sobre los costos brutos de toda la generación, no sobre un valor absoluto.

#### 4.4 Algoritmo genético

**crear\_individuo()** devuelve un diccionario `{"t": random.random()}`, es decir, un individuo nuevo con su único gen (`t`) inicializado de forma uniforme en  $[0, 1]$ .

**evaluar\_poblacion(poblacion)** es el corazón del fitness. Primero traduce el `t` de cada individuo a un punto real con `interpolate_point`. Calcula el `costo_construccion_bruto` de todos los puntos y los normaliza juntos con `normalizar` (por eso `f_costo` es relativo a la generación actual, no a un valor absoluto). Después, para cada individuo, llama a `evaluar_restriccion_poblacional`, combina `f_costo` y `f_poblacional` con los pesos de `PESOS_FITNESS` (0,5 y 0,5), y si `cumple_restriccion_dura` es `False` multiplica el resultado por `PENALIZACION_RESTRICCION_DURA` (0,02). Devuelve dos listas paralelas: `fitness_total` (un número por individuo) y `detalle` (un diccionario por individuo con el punto, los costos, las densidades y los puntajes parciales, que se reutiliza para el reporte final y el mapa).

**seleccion\_torneo(poblacion, fitness, k)** implementa selección por torneo: toma `k` individuos al azar de la población (junto con su fitness, con `random.sample`), los ordena de mayor a menor fitness y devuelve el primero. Con `k = TAM_TORNEO = 2`, en cada torneo compiten dos individuos y gana el de mayor fitness. Se llama dos veces por cada par de hijos que hay que generar, una para cada padre.

**cruza(p1, p2)** recibe dos padres y, con probabilidad `PROB_CRUZA` (0,75), genera dos hijos por combinación lineal (cruza aritmética): elige un  $\alpha$  aleatorio en  $[0, 1]$  y calcula  $t1 = \alpha \cdot p1.t + (1-\alpha) \cdot p2.t$  y  $t2 = \alpha \cdot p2.t + (1-\alpha) \cdot p1.t$ , acotando ambos a  $[0, 1]$ . Si el sorteo de probabilidad no favorece la cruce, devuelve copias idénticas de los padres sin combinar.

**mutacion(ind, sigma)** con probabilidad `PROB_MUTACION` (0,25), le suma al `t` del individuo un valor aleatorio de una normal con media 0 y desvío `sigma` (`random.gauss(0, sigma)`), y vuelve a acotar el resultado a  $[0, 1]$ . En este proyecto `sigma` es fijo (`SIGMA_MUTACION = 0,25`): al ser relativamente grande en una escala de  $[0, 1]$ , la mutación puede desplazar a un individuo a una zona muy distinta del corredor en un solo paso, lo que favorece la exploración pero también puede hacer que el mejor individuo de una generación "salte" a una región peor en la siguiente.

**correr\_ga()** es el ciclo evolutivo completo. Crea la población inicial con `crear_individuo()`, y por cada una de las `N_GENERACIONES` (100) generaciones: evalúa la población con `evaluar_poblacion`; identifica el mejor individuo de la generación (`np.argmax(fitness)`) y, si supera al mejor histórico guardado en `mejor_fitness_global`, actualiza `mejor_global` (este seguimiento del mejor global es independiente del reemplazo generacional, y es lo que garantiza que el resultado final no se pierda aunque una generación completa tenga fitness bajo); guarda en las listas `historia_mejor`, `historia_promedio`, `historia_peor` e `historia_std` las estadísticas de fitness de la generación, para el gráfico de convergencia; y arma la siguiente generación repitiendo selección por torneo, cruza y mutación hasta juntar de nuevo `POP_SIZE` (60) individuos. El reemplazo es generacional puro: la población entera se reemplaza y no hay elitismo dentro del ciclo (el único "resguardo" del mejor resultado es la variable `mejor_global`, que se reporta al final). Devuelve `mejor_global` (con su detalle), las cuatro historias de fitness y la población de la última generación.

#### 4.5 Reporte de resultados

**graficar\_convergencia(...)** recibe las cuatro historias que devuelve `correr_ga` y arma, con `matplotlib`, el gráfico de líneas de mejor/promedio/peor fitness por generación, con una banda sombreada de  $\pm 1$  desvío estándar alrededor del promedio y un recuadro de texto con los valores finales. Guarda el resultado como PNG en la ruta indicada.

**generar\_mapa(mejor, poblacion\_final, fitness\_total, path)** arma el mapa interactivo con `folium`: crea el mapa base centrado en el punto óptimo, agrega la polilínea del corredor, las ciudades, estaciones transformadoras, puertos y aeropuertos como marcadores, dibuja como puntos grises la posición de cada individuo de la última generación (para visualizar en qué parte del corredor terminó concentrada la búsqueda), traza los dos círculos de restricción (2 y 10 millas) alrededor del punto óptimo, y agrega el marcador final con un popup que resume el detalle del mejor individuo (costo, densidades, fitness). Las funciones auxiliares `_bounds_de_interes`, `_generar_overlay_densidad_ghs_pop` y `_leyenda_densidad_ghs_pop_html` (descriptas al usarlas dentro de esta) preparan, respectivamente, el recuadro geográfico de interés, la reproyección del raster GHS-POP a Web Mercator para superponerlo como imagen sobre el mapa, y la leyenda de colores de esa capa.

### 5. Especificación técnica e infraestructura

#### 5.1 Software

El prototipo está escrito en Python 3 (Específicamente 3.14.4) y utiliza las siguientes bibliotecas:

- `numpy`: muestreo de radios y ángulos, estadísticas por generación. (V2.5.2)
- `rasterio` y `pyproj`: lectura del mosaico GHS-POP, transformación de coordenadas y reproyección. (V 1.5.1 - V3.7.2)
- `folium` (Leaflet) y `Pillow`: mapa HTML interactivo y overlay de densidad. (V0.20.0 - V12.3.0)
- `matplotlib`: gráfico de convergencia. (V3.11.1)
- `math`, `random`, `json`, `glob`, `os` y `datetime`, de la biblioteca estándar.

#### 5.2 Hardware e infraestructura

| Componente        | Especificación       |
|-------------------|----------------------|
| Equipo            | MacBook Air          |
| Procesador        | Apple M4             |
| Memoria RAM       | 16 GB                |
| Sistema operativo | macOS Sequoia 15.7.9 |

Tabla 4. Entorno de ejecución

### 5.3 Ejecución

Con los tiles GHS-POP en `ghs_pop_tiles/` y las dependencias instaladas, se ejecuta con `python ga_central_nuclear.py`. Cada corrida usa una semilla aleatoria distinta; para reproducir una corrida exacta se reemplazan las llamadas a `random.seed()` y `np.random.seed()` en el bloque principal por una semilla fija.

### 6. Salidas del prototipo

Cada corrida genera, en la carpeta `resultados_ga/` y con marca de tiempo para no pisar corridas anteriores:

- `mapa_optimo_<fecha>.html`: mapa interactivo con la densidad GHS-POP, el corredor, ciudades, estaciones transformadoras, puertos y aeropuertos, la posición de la población final, los anillos de 2 y 10 millas y el punto óptimo con su detalle.
- `convergencia_ga_<fecha>.png`: evolución del mejor, el promedio y el peor fitness, con banda de  $\pm 1$  desvío estándar.
- `resultado_optimo_<fecha>.json`: coordenadas, costo, densidades y fitness del mejor individuo.
- `log_corridas.csv`: una fila por corrida, para comparar resultados entre ejecuciones.

### 7. Resultados y verificación

La verificación consiste en comprobar que el punto elegido cumpla la restricción dura, que el algoritmo converja y que los resultados sean consistentes entre corridas. Los valores de la última corrida generada son los siguientes:

| Indicador                                                     | Valor                                                    |
|---------------------------------------------------------------|----------------------------------------------------------|
| Posición t del punto óptimo                                   | 0,8730                                                   |
| Coordenadas (lat, lon)                                        | -32,5066, -60,8168 (sobre el corredor, cerca de Rosario) |
| Costo de construcción estimado (USD)                          | USD 13.500.460                                           |
| Densidad máxima 0-2 millas (hab/km <sup>2</sup> )             | 19,3 hab/km <sup>2</sup>                                 |
| Densidad máxima 2-10 millas (hab/km <sup>2</sup> )            | 374,2 hab/km <sup>2</sup>                                |
| ¿Cumple la restricción dura? (umbral 25 hab/km <sup>2</sup> ) | Sí                                                       |
| Fitness costo / poblacional / total                           | 0,871 / 1,000 / 0,935                                    |

Tabla 5. Resultado de la corrida presentada

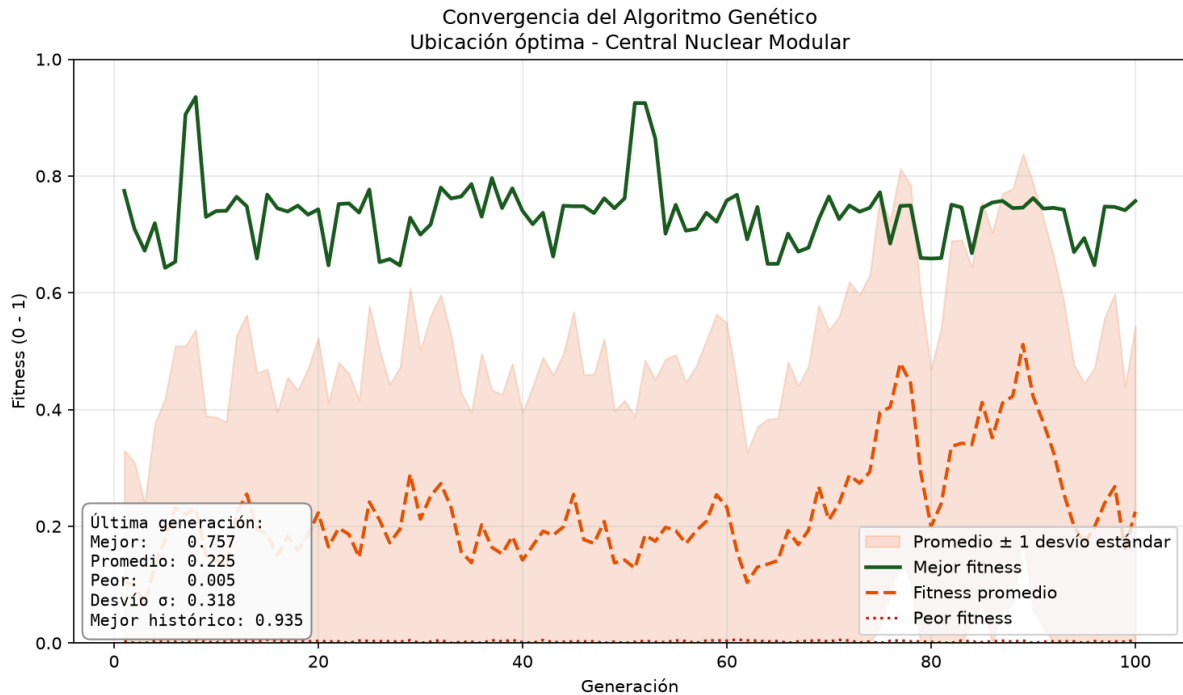


Figura 1. Convergencia del algoritmo genético (mejor, promedio y peor fitness por generación)

El punto óptimo cae sobre el tramo del corredor cercano a Rosario, con densidad muy por debajo del umbral en el anillo de 2 millas (19,3 frente a 25 hab/km<sup>2</sup>) pero cerca del umbral en el anillo de 2-10 millas (374,2 frente a 400 hab/km<sup>2</sup>), lo que explica que  $f_{\text{poblacional}}$  dé 1,000: ambos anillos están dentro de lo tolerado. El fitness de costo (0,871) indica que, dentro de esa generación, el sitio elegido estuvo entre los más baratos de construir.

El gráfico de convergencia muestra que el mejor fitness no crece de forma monótona: oscila entre valores cercanos a 0,8. Esta variabilidad es consistente con que el reemplazo es puramente generacional y sin elitismo (sección 2.3): el mejor individuo no pasa automáticamente a la siguiente generación, y solo se conserva por separado en la variable `mejor_global`.

### Relación con los objetivos del proyecto

| Objetivo específico                                                                      | Tratamiento en el prototipo                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Evaluar el riesgo sísmico de cada ubicación                                              | Analizado en el marco teórico (Zona 0 de INPRES-CIRSOC 103 en todo el corredor). Se descartó como variable del fitness porque no diferencia un punto de otro.                                                                 |
| Relevar la infraestructura eléctrica y estimar el costo adicional donde sea insuficiente | Implementado: <code>costo_tendido_electrico</code> calcula el costo de la línea hasta la estación transformadora más cercana.                                                                                                 |
| Calcular la distancia a las ciudades seleccionadas                                       | Se abordó mediante la densidad poblacional por anillos sobre GHS-POP, que mide la población efectivamente expuesta en lugar de la distancia a un punto único por ciudad. Las ciudades se muestran como referencia en el mapa. |

*Tabla 6. Objetivos y su tratamiento en el modelo*

## **8. Limitaciones y trabajo futuro**

- Los costos unitarios de tendido y transporte son valores de referencia. Deberían reemplazarse por cotizaciones reales de obras de alta tensión y de logística de cargas pesadas, por ejemplo con datos de CAMMESA.
- Las distancias a estaciones transformadoras y puertos se miden en línea recta (Haversine). No consideran el trazado real de la línea ni la ruta fluvial de las barcazas.
- El costo se normaliza dentro de cada generación, por lo que  $f_{\text{costo}}$  es relativo a la población de esa generación y no es comparable de forma absoluta entre generaciones.
- Los pesos del fitness (0,5/0,5 y 0,65/0,35) y los umbrales de densidad son decisiones del grupo. Conviene analizar la sensibilidad del resultado a estos valores, como hacen varios de los trabajos revisados con distintos escenarios de pesos.
- La resolución de GHS-POP es de 1 km por celda, lo que limita el detalle del muestreo dentro del anillo de 2 millas (unos 3,2 km de radio).
- El modelo optimiza un único sitio. Una extensión natural es decidir simultáneamente cuántos sitios y cuántos módulos, y agregar criterios como disponibilidad de agua de refrigeración, uso del suelo y aceptación social.
- No se incorporó el riesgo sísmico; si se extendiera el corredor hacia el oeste del país, volvería a ser una variable relevante.

## **Referencias**

[1] Comisión Europea, JRC. GHS-POP R2023A,  
GHS\_POP\_E2030\_GLOBE\_R2023A\_54009\_1000\_V1\_0.  
[https://human-settlement.emergency.copernicus.eu/ghs\\_pop2023.php](https://human-settlement.emergency.copernicus.eu/ghs_pop2023.php)

[2] Marco teórico del Trabajo Práctico Integrador (primera parte), Grupo 14, Comisión 303, 2026, y las fuentes allí citadas.

## **Anexo: Código del Trabajo Práctico**

<https://github.com/TomasRibotta20/Algoritmos/tree/main/TpIntegrador>